TY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT TEMPLATE

1. Student Details

Student Name	Atharv Jeevan Fatale
PRN	202401070076
Branch	ENTC
Division	A
Batch	A2
Roll No.	202401070076
Date of Submission	22-10-2026

2. Practical Details

Practical No.	4
Practical Title	Classification Models (SVM, Decision Tree, KNN and Naïve Bayes)
Date Performed	22-10-2026
Faculty Name	Mrs. Anita Shingade

3. Problem Statement

To develop classification models using Support Vector Machine (SVM), Decision Tree, K-Nearest Neighbours (KNN) and Naïve Bayes on a real-world medical dataset (Breast Cancer Wisconsin – Diagnostic) to predict whether a tumour is malignant or benign. The models are to be evaluated using Accuracy, Precision, Recall, F1-Score and Loss (Log Loss), supported by confusion matrices, ROC curves and cross-validation, and the findings are to be compared in order to discuss the suitability of each algorithm for different classification tasks.

4. Objectives

1. To develop SVM, Decision Tree, KNN and Naïve Bayes classifiers for a real-world binary classification problem.
2. To preprocess the data correctly (stratified splitting and feature scaling inside a pipeline to avoid data leakage).
3. To tune the key hyper-parameters of every model using cross-validated grid search.
4. To evaluate the models using Accuracy, Precision, Recall, F1-Score and Log Loss, and to support them with the confusion matrix, ROC-AUC and k-fold cross-validation.
5. To summarise the comparative analysis and discuss the suitability of each algorithm for various classification tasks.

5. Expected Learning Outcomes

After completing this practical, the student will be able to:

1. Explain the working principle of SVM, Decision Tree, KNN and Naïve Bayes classifiers.
2. Build and tune classification models on real-world data using scikit-learn pipelines.
3. Evaluate and compare classifiers using Accuracy, Precision, Recall, F1-Score and Log Loss.
4. Analyse over-fitting, error types and trade-offs, and select an algorithm that suits a given classification task.

6. Theory / Concept

Classification

Classification is a supervised learning task in which a model learns a mapping from input features to a discrete class label. In this practical the target is binary: 0 = Malignant and 1 = Benign. The positive class used for Precision, Recall and F1 is therefore Benign (label 1); malignant-class recall is reported separately because missing a malignant tumour is the more serious error.

Support Vector Machine (SVM)

SVM finds the hyperplane $w \cdot x + b = 0$ that maximises the margin between the two classes. Soft-margin training minimises $\frac{1}{2}\|w\|^2 + C \sum \max(0, 1 - y_i(w \cdot x_i + b))$ (hinge loss). C controls the margin/error trade-off; kernels (linear, RBF) allow non-linear boundaries, with γ controlling the RBF width. SVM is distance-based, so features must be scaled.

Decision Tree

A Decision Tree recursively splits the data on the feature and threshold that give the largest impurity reduction. Gini $= 1 - \sum p_k^2$ and Entropy $= -\sum p_k \log_2 p_k$. Trees are interpretable and need no scaling, but deep trees over-fit; max_depth and min_samples_leaf control complexity.

K-Nearest Neighbours (KNN)

KNN is a lazy, instance-based learner: a new sample is assigned the majority class among its k closest training samples (Euclidean $d = \sqrt{\sum(x_i - y_i)^2}$ or Manhattan distance). Small k gives low bias/high variance, large k gives smoother boundaries. It needs scaled features and becomes slow on large datasets.

Gaussian Naïve Bayes

Naïve Bayes applies Bayes' theorem, $P(C|x) \propto P(C) \prod P(x_i|C)$, assuming features are conditionally independent given the class. Gaussian NB models each $P(x_i|C)$ as a normal distribution. It is extremely fast and works with little data, but correlated features violate its independence assumption.

Evaluation Metrics

- Accuracy: (TP + TN) / (TP + TN + FP + FN)
- Precision: TP / (TP + FP)
- Recall: TP / (TP + FN)
- F1-Score: 2 · Precision · Recall / (Precision + Recall)
- Log Loss (loss): $-(1/n) \sum [\, y_i \log p_i + (1 - y_i) \log(1 - p_i) \,]$ – penalises confident wrong probabilities; lower is better
- ROC-AUC: Area under the TPR-vs-FPR curve; threshold-independent ranking quality

Application

Breast Cancer Wisconsin (Diagnostic) dataset: 30 numerical features computed from digitised images of fine-needle-aspirate cell nuclei (mean, standard error and worst value of radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry and fractal dimension).

Feature Group	Type	Use
30 nuclei measurements (mean / error / worst)	Numerical	Input
target (0 = malignant, 1 = benign)	Categorical (binary)	Target

This is a binary classification problem because the target takes one of two discrete values. All four algorithms are used because they represent four different learning families (margin-based, rule-based, instance-based and probabilistic), which makes the comparison meaningful.

7. Algorithm / Pseudocode

Classification –

START
↓
Load Breast Cancer Wisconsin dataset

↓
Inspect shape, class balance and missing values
↓
Separate features (X) and target (y)
↓
Stratified train–test split (75 % / 25 %)
↓
Build a pipeline per model (StandardScaler for SVM and KNN)
↓
Tune hyper-parameters with 5-fold stratified GridSearchCV (scoring = F1)
↓
Train the best estimator of SVM, Decision Tree, KNN and Naïve Bayes
↓
Predict labels and class probabilities on the test set
↓
Calculate Accuracy, Precision, Recall, F1-Score and Log Loss
↓
Draw confusion matrices and ROC curves; run 5-fold cross-validation
↓
Compare models and analyse suitability
↓
STOP

8. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Python	Programming
Google Colab	Development environment
Pandas	Data handling and result tables
NumPy	Numerical computation
Scikit-learn	Dataset, preprocessing, SVM / Decision Tree / KNN / Naïve Bayes, GridSearchCV, cross-validation and metrics
Matplotlib	Visualization (metric bars, confusion matrices, ROC, validation curves)

Dataset / Resources:

- 569 records (212 malignant, 357 benign)
- 30 numerical feature columns + 1 target column
- No missing values; loaded with sklearn.datasets.load_breast_cancer()
- Target: diagnosis (0 = malignant, 1 = benign)
- Split: 426 training samples, 143 testing samples (stratified, random_state = 42)

9. Implementation / GitHub Repository

Particular	Details
GitHub Repository URL	**https://github.com/Atharvfatale/Classification-Models-SVM-Decision-Tree-KNN-Naive-Bayes-.git**
Repository Name	**Classification Models (SVM, Decision Tree, KNN, Naive Bayes)**
Branch Used	**main**
Commit ID / Latest Commit	**0ec22a8092bf93c9e90f7179f24a370c38bdc8e8**

Particular	Details
Date of Last Commit	22-10-2026

10. Test Cases / Input and Output

Test Case	Input	Expected Output	Actual Output	Status
TC1	Breast Cancer Wisconsin dataset	569 × 30 features, no missing values	Shape (569, 30); 0 missing values; 357 benign / 212 malignant	Pass
TC2	Stratified 75/25 split	Class proportions preserved	426 training and 143 test samples	Pass
TC3	Grid-search and train SVM	Best model selected	Best: C=0.1, gamma=scale, kernel=linear; test accuracy 0.9860	Pass
TC4	Grid-search and train Decision Tree	Best model selected	Best: criterion=entropy, max_depth=4, min_samples_leaf=3; test accuracy 0.9161	Pass
TC5	Grid-search and train KNN	Best model selected	Best: metric=euclidean, n_neighbors=9, weights=uniform; test accuracy 0.9650	Pass
TC6	Grid-search and train Naïve Bayes	Best model selected	Best: var_smoothing=1e-09; test accuracy 0.9371	Pass
TC7	Evaluate all models	Accuracy, Precision, Recall, F1, Log Loss generated	All metrics generated for the four models (Section 11)	Pass
TC8	Confusion matrix, ROC and 5-fold CV	Plots and CV scores generated	Four confusion matrices, ROC curves and CV mean ± std generated	Pass

11. Results / Observations

All values below are measured on the 143-sample held-out test set with the tuned models (positive class = Benign).

Model	Accuracy	Precision	Recall	F1-Score	Log Loss	ROC-AUC
SVM	0.9860	0.9889	0.9889	0.9889	0.0778	0.9964
Decision Tree	0.9161	0.9432	0.9222	0.9326	1.3325	0.9498
KNN	0.9650	0.9474	1.0000	0.9730	0.1029	0.9930
Naive Bayes	0.9371	0.9263	0.9778	0.9514	0.3671	0.9893

Confusion matrix counts (test set)

Model	TN (malignant correct)	FP (malignant → benign)	FN (benign → malignant)	TP (benign correct)	Malignant recall
SVM	52	1	1	89	0.9811
Decision Tree	48	5	7	83	0.9057
KNN	48	5	0	90	0.9057
Naive Bayes	46	7	2	88	0.8679

Generalisation and cross-validation

Model	Baseline Test Acc (default params)	Tuned Test Acc	Train Acc	5-fold CV Acc (mean ± std)	Train − CV gap

SVM	0.9790	0.9860	0.9836	0.9766 ± 0.0128	0.0070
Decision Tree	0.9231	0.9161	0.9742	0.9390 ± 0.0156	0.0352
KNN	0.9790	0.9650	0.9695	0.9695 ± 0.0094	0.0000
Naive Bayes	0.9371	0.9371	0.9460	0.9413 ± 0.0333	0.0047

Computation and selected hyper-parameters

Model	Tuning + fit time (s)	Predict time, 143 samples (s)	Best hyper-parameters
SVM	3.91	0.0031	C=0.1, gamma=scale, kernel=linear
Decision Tree	3.67	0.0023	criterion=entropy, max_depth=4, min_samples_leaf=3
KNN	2.70	0.0046	metric=euclidean, n_neighbors=9, weights=uniform
Naive Bayes	0.30	0.0027	var_smoothing=1e-9

Timings were measured on a cloud CPU and vary between machines and runs; the grid sizes also differ per model (SVM 24, Decision Tree 48, KNN 28, Naïve Bayes 4 combinations × 5 folds), so they indicate only rough relative cost.

Observations

- SVM gave the best result on every metric except recall: Accuracy 0.9860, F1 0.9889, lowest Log Loss (0.0778) and highest ROC-AUC (0.9964). It made only 2 errors out of 143 test samples.
- KNN achieved a perfect benign recall of 1.0000 but misclassified 5 malignant tumours as benign (malignant recall 0.9057), so its high recall comes with lower precision (0.9474).
- Naïve Bayes was by far the fastest to train and reached Accuracy 0.9371, but it produced the most false “benign” calls on malignant tumours (7).
- Decision Tree was the weakest: Accuracy 0.9161, ROC-AUC 0.9498 and a very high Log Loss of 1.3325. Its training accuracy (0.9742) is much higher than its cross-validated accuracy (0.9390), the largest generalisation gap of all models.
- The ranking SVM > KNN > Naïve Bayes > Decision Tree is the same in cross-validation and on the test set, which makes it more trustworthy than a single split.
- With only 143 test samples one prediction equals about 0.7 percentage points, so small differences (for example SVM vs KNN) should not be over-interpreted.

Screenshots / Graphs / Output:

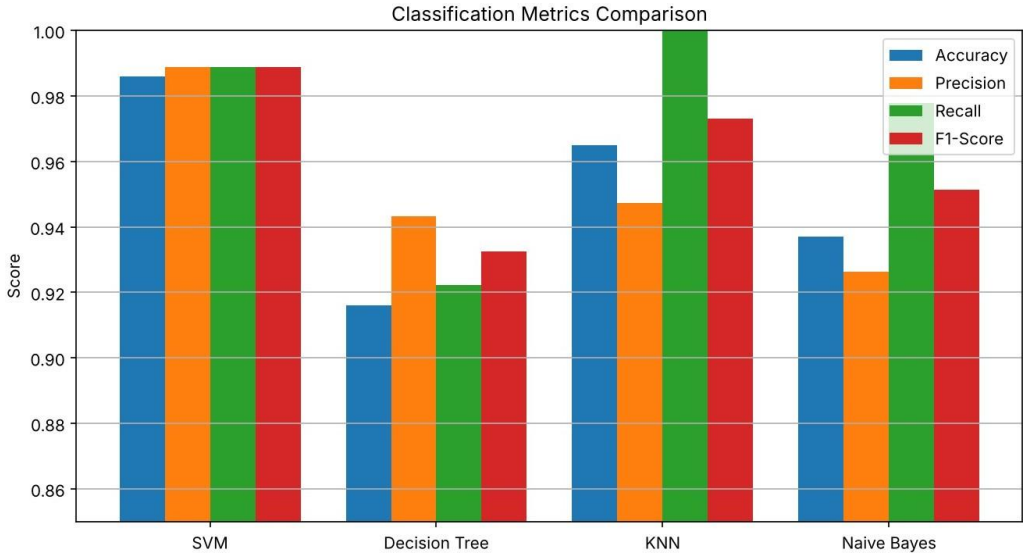


Figure 1: Accuracy, Precision, Recall and F1-Score of the four models

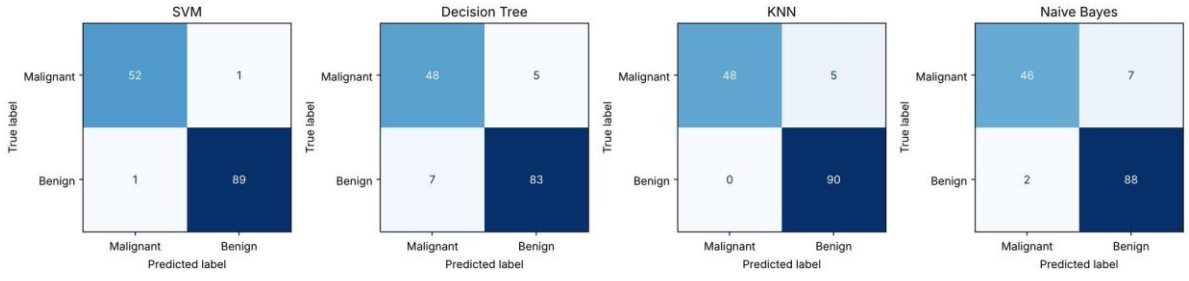


Figure 2: Confusion matrices on the test set (rows = actual, columns = predicted)

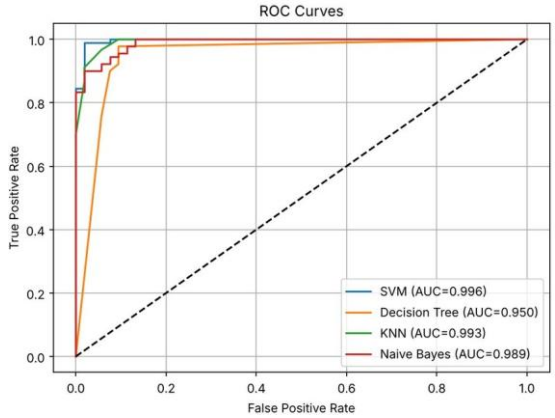


Figure 3: ROC curves with AUC

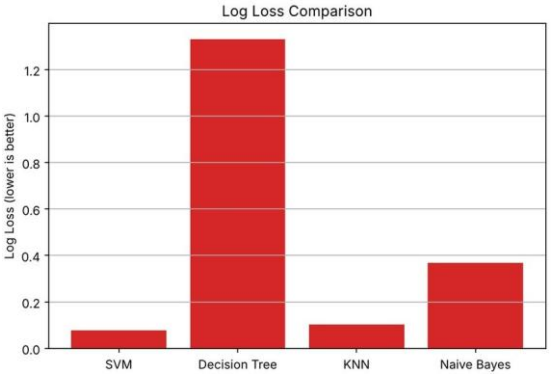


Figure 4: Log Loss (lower is better)

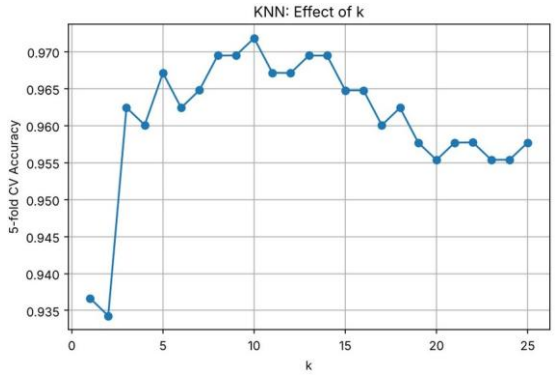


Figure 5: KNN – effect of k (5-fold CV accuracy)

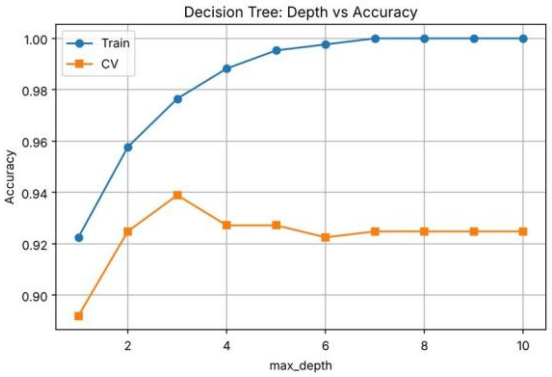


Figure 6: Decision Tree – depth vs accuracy

Console output of the program (abridged):

Dataset Shape: (569, 30)
Missing values: 0
Class counts (0=malignant,1=benign): {1: 357, 0: 212}
Training Samples: 426 Testing Samples: 143

SVM: best params = {'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'} Acc=0.9860 P=0.9889 R=0.9889 F1=0.9889
LogLoss=0.0778 AUC=0.9964

Decision Tree: best params = {'criterion': 'entropy', 'max_depth': 4, 'min_samples_leaf': 3} Acc=0.9161 P=0.9432 R=0.9222 F1=0.9326
LogLoss=1.3325 AUC=0.9498

KNN: best params = {'metric': 'euclidean', 'n_neighbors': 9, 'weights': 'uniform'} Acc=0.9650 P=0.9474 R=1.0000 F1=0.9730
LogLoss=0.1029 AUC=0.9930

Naive Bayes: best params = {'var_smoothing': 1e-09}
Acc=0.9371 P=0.9263 R=0.9778 F1=0.9514 LogLoss=0.3671 AUC=0.9893

12. Performance Evaluation / Analysis

Criterion	SVM	Decision Tree	KNN	Naïve Bayes
Test accuracy	0.9860	0.9161	0.9650	0.9371
F1-Score	0.9889	0.9326	0.9730	0.9514
Log Loss	0.0778	1.3325	0.1029	0.3671
Malignant tumours missed (of 53)	1	5	5	7
Criterion	**SVM**	**Decision Tree**	**KNN**	**Naïve Bayes**

Needs feature scaling	Yes	No	Yes	No
Interpretability	Medium (high for linear kernel)	High (rules)	Medium	Medium–High
Training cost	Medium	Low	None (lazy)	Very low
Prediction cost	Low	Very low	Higher (distance to all samples)	Very low
Over-fitting tendency	Low–Medium	High	Medium (depends on k)	Low
Probability quality (Log Loss)	Good	Poor	Good	Moderate

Discussion

- SVM: The best-performing model here. The linear kernel with a small C=0.1 was selected, which suggests the two classes are almost linearly separable in the 30-dimensional scaled space; strong regularisation prevented over-fitting.
- Decision Tree: A single tree is high-variance. Training accuracy keeps rising with depth to 1.00 while cross-validated accuracy peaks near depth 3 (Figure 6), i.e. deeper trees only memorise noise. Its hard, near 0/1 leaf probabilities make every confident mistake costly, which explains the large Log Loss of 1.33 despite 91.6 % accuracy.
- KNN: Performs well after scaling; accuracy is stable for k between about 5 and 14 (Figure 5) and falls for very small k (noisy) and large k (over-smoothed). Its perfect benign recall is only valuable if the cost of a false "malignant" is higher than a false "benign", which is not the case in this medical setting.
- Naïve Bayes: Accurate enough and by far the cheapest, even though the 30 features are strongly correlated (e.g. radius, perimeter and area), which violates its independence assumption. Its Log Loss (0.367) is higher because independence makes probabilities over-confident.
- Accuracy alone would hide an important point: the models differ in which error they make. For cancer screening a false "benign" (missed malignancy) is far more harmful than a false alarm, so malignant recall (SVM 0.981, Decision Tree 0.906, KNN 0.906, Naïve Bayes 0.868) should drive model choice rather than accuracy.
- Hyper-parameter tuning was selected on cross-validated F1, not on the test set. It did not improve every model on the test set (KNN default k = 5 scored 0.9790 vs 0.9650 tuned), which shows the gap is within the noise of a 143-sample test set and that a single split cannot rank models reliably.

Suitability of each algorithm for different classification tasks

Algorithm	Best suited for	Avoid / limitations
SVM	Medium-sized, high-dimensional numerical data (medical, text, bioinformatics); clear margin between classes	Very large datasets (training scales poorly); needs scaling; less interpretable with non-linear kernels
Decision Tree	Tasks needing explainable rules, mixed numeric/categorical data, no scaling (credit rules, triage rules); base learner for Random Forest / boosting	Noisy, small data when used as a single deep tree (over-fits, unstable, poor probabilities)
KNN	Small–medium datasets with meaningful distance (recommendation, similarity search, quick baselines)	Large data or many irrelevant features (slow, curse of dimensionality); sensitive to scaling
Naïve Bayes	Very high-dimensional sparse data and real-time use (spam filtering, text classification, small training sets)	Strongly correlated features; probability estimates are over-confident

Ethical and societal considerations

- This is a decision-support experiment, not a diagnostic tool; a deployed medical model would need clinical validation on data from several hospitals, because this dataset comes from a single source and may not represent all patient groups.
- False negatives (missed malignancies) are the dangerous error, so a real system would tune the decision threshold to maximise malignant recall and keep a clinician in the loop.
- Patient data requires consent, anonymisation and secure handling; this practical uses a public, de-identified dataset.

Limitations and suggested improvements

- Use repeated or nested cross-validation to obtain reliable confidence intervals instead of one 143-sample test split.
- Calibrate probabilities (CalibratedClassifierCV) and tune the decision threshold for malignant recall.
- Apply feature selection or PCA to reduce correlated features, which should help Naïve Bayes and KNN.
- Compare ensemble methods (Random Forest, Gradient Boosting) that fix the single-tree variance problem.
- Validate on an external dataset to test generalisation.

13. Conclusion

SVM, Decision Tree, KNN and Naïve Bayes classifiers were developed on the Breast Cancer Wisconsin dataset using leakage-free pipelines and cross-validated hyper-parameter tuning, and were evaluated using Accuracy, Precision, Recall, F1-Score, Log Loss, ROC-AUC and confusion matrices.

SVM was the best model (Accuracy 0.9860, F1 0.9889, Log Loss 0.0778, ROC-AUC 0.9964), followed by KNN (0.9650), Naïve Bayes (0.9371) and Decision Tree (0.9161). The same order appeared in cross-validation, and SVM also missed the fewest malignant tumours (1 of 53), which is the decisive criterion for a medical task. The single Decision Tree over-fitted and produced poorly calibrated probabilities, while Naïve Bayes offered the best speed-to-accuracy ratio.

No algorithm is universally best: SVM suits high-dimensional numeric data, Decision Trees suit explainable rule-based decisions, KNN suits small datasets and quick baselines, and Naïve Bayes suits fast, high-dimensional or text problems. The choice should be guided by the data, the cost of each error type and the need for interpretability, not by accuracy alone.

14. Assignment Questions

Q1. Why must features be scaled for SVM and KNN but not for Decision Trees and Naïve Bayes?

Answer: SVM and KNN depend on distances between samples, so a feature with a large numeric range (e.g. area ≈ 1000) would dominate one with a small range (e.g. smoothness ≈ 0.1). Decision Trees split on one feature at a time and Gaussian Naïve Bayes models each feature separately, so neither is affected by feature scale.

Q2. Why is Recall more important than Accuracy in tumour classification?

Answer: Accuracy treats all errors equally. A missed malignant tumour (false negative) can delay treatment, while a false alarm only leads to an extra test. Recall on the malignant class shows how many cancers the model catches: SVM 0.981 vs Naïve Bayes 0.868 in this experiment, a difference that accuracy (0.986 vs 0.937) understates.

Q3. What does Log Loss measure and why is it high for the Decision Tree?

Answer: Log Loss measures the quality of predicted probabilities by penalising confident wrong predictions heavily. A tree outputs the class proportion of a leaf, which is often 0 or 1, so each misclassification is very costly (Log Loss 1.33), whereas SVM and KNN give smoother probabilities (0.078 and 0.103).

Q4. How does the value of k affect KNN?

Answer: Small k follows noise (low bias, high variance; CV accuracy 0.937 at k = 1), large k over-smooths the boundary (0.958 at k = 25). Accuracy was highest around k = 8–14 in cross-validation (about 0.97).

15. Implementation / Source Code

```
# Practical 4: Classification Models (SVM, Decision Tree, KNN, Naive Bayes) import
warnings; warnings.filterwarnings("ignore")
# Dataset: Breast Cancer Wisconsin (Diagnostic) - bundled with scikit-learn import
json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import (train_test_split, GridSearchCV,
                                     StratifiedKFold, cross_val_score)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler from
sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier from
sklearn.neighbors import KNeighborsClassifier from
sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, log_loss, roc_auc_score, roc_curve,
                             confusion_matrix, ConfusionMatrixDisplay)
import time

RS = 42
OUT = ""    # figures are saved in the current Colab folder

# 1. LOAD DATA
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target                    # target: 0 = malignant, 1 = benign
print("Dataset Shape:", X.shape)
print("Missing values:", int(X.isnull().sum().sum()))
print("Class counts (0=malignant,1=benign):", y.value_counts().to_dict())

# 2. STRATIFIED TRAIN-TEST SPLIT
X_train, X_test, y_train, y_test = train_test_split( X, y,
    test_size=0.25, stratify=y, random_state=RS)
print("Training Samples:", len(X_train), " Testing Samples:", len(X_test))

# 3. MODELS + HYPERPARAMETER GRIDS (scaler inside pipeline -> no data
leakage) cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RS)
specs = {
    "SVM": (Pipeline([("sc", StandardScaler()),
                      ("clf", SVC(probability=True, random_state=RS))]),
            {"clf__C": [0.1, 1, 10, 100],
             "clf__kernel": ["linear", "rbf"],
             "clf__gamma": ["scale", 0.01, 0.1]}),
    "Decision Tree": (Pipeline([("clf", DecisionTreeClassifier(random_state=RS))]),
                      {"clf__max_depth": [2, 3, 4, 5, 7, None],
                       "clf__min_samples_leaf": [1, 3, 5, 10], "clf__
```

```
                                criterion": ["gini", "entropy"]}),
    "KNN": (Pipeline([("sc", StandardScaler()),
                                ("clf", KNeighborsClassifier())]),
                {"clf__n_neighbors": [1, 3, 5, 7, 9, 11, 15], "clf__
                 weights": ["uniform", "distance"], "clf__
                 metric": ["euclidean", "manhattan"]}),
    "Naive Bayes": (Pipeline([("clf", GaussianNB())]),
                            {"clf__var_smoothing": [1e-11, 1e-9, 1e-7, 1e-5]}),
}

results, best_models, best_params, cm_store, roc_store = [], {}, {}, {}, {}
default_models = {
    "SVM": Pipeline([("sc", StandardScaler()), ("clf", SVC(probability=True, random_state=RS))]),
    "Decision Tree": DecisionTreeClassifier(random_state=RS),
    "KNN": Pipeline([("sc", StandardScaler()), ("clf", KNeighborsClassifier())]), "Naive
    Bayes": GaussianNB(),
}
baseline = {}
for name, m in default_models.items(): m.fit(X_train,
    y_train)
    baseline[name] = accuracy_score(y_test, m.predict(X_test))

for name, (pipe, grid) in specs.items():
    gs = GridSearchCV(pipe, grid, cv=cv, scoring="f1", n_jobs=1)
    t0 = time.time(); gs.fit(X_train, y_train); fit_t = time.time() - t0 model =
    gs.best_estimator_
    t0 = time.time(); pred = model.predict(X_test); pred_t = time.time() - t0 proba =
    model.predict_proba(X_test)
    cvs = cross_val_score(model, X_train, y_train, cv=cv, scoring="accuracy") tr_acc
    = accuracy_score(y_train, model.predict(X_train))
    row = {
        "Model": name,
        "Accuracy": accuracy_score(y_test, pred), "Precision":
        precision_score(y_test, pred), "Recall":
        recall_score(y_test, pred),
        "F1-Score": f1_score(y_test, pred), "Log
        Loss": log_loss(y_test, proba),
        "ROC-AUC": roc_auc_score(y_test, proba[:, 1]),
        "CV Acc (mean)": cvs.mean(), "CV Acc (std)": cvs.std(),
        "Train Acc": tr_acc,
        "Fit+Tune Time (s)": fit_t, "Predict Time (s)": pred_t,
        "Baseline Acc (default params)": baseline[name],
    }
    results.append(row)
    best_models[name] = model
    best_params[name] = {k.replace("clf__", ""): (v if not isinstance(v, np.generic) else v.item()) for k, v in
                         gs.best_params_.items()}
    cm_store[name] = confusion_matrix(y_test, pred)
    fpr, tpr, _ = roc_curve(y_test, proba[:, 1]); roc_store[name] = (fpr, tpr) print(f"\n{name}: best
```

```
    params = {best_params[name]}")
    print(f"  Acc={row['Accuracy']:.4f} P={row['Precision']:.4f} R={row['Recall']:.4f} "
          f"F1={row['F1-Score']:.4f} LogLoss={row['Log Loss']:.4f} AUC={row['ROC-AUC']:.4f}")

res = pd.DataFrame(results)
print("\n" + "=" * 60 + "\nFINAL COMPARISON\n" + "=" * 60)
print(res.round(4).to_string(index=False))

# 4. PLOTS
names = list(best_models) #
4a. metric comparison
fig, ax = plt.subplots(figsize=(9, 5)) w =
0.2; xs = np.arange(len(names))
for i, met in enumerate(["Accuracy", "Precision", "Recall", "F1-Score"]): ax.bar(xs + i
    * w, res[met], w, label=met)
ax.set_xticks(xs + 1.5 * w); ax.set_xticklabels(names); ax.set_ylim(0.85, 1.0) ax.set_ylabel("Score");
ax.set_title("Classification Metrics Comparison"); ax.legend(); ax.grid(axis="y")
plt.tight_layout(); plt.savefig(OUT + "metrics.png", dpi=150); plt.show() # 4b.
confusion matrices
fig, axes = plt.subplots(1, 4, figsize=(15, 3.6)) for a, n in
zip(axes, names):
    ConfusionMatrixDisplay(cm_store[n], display_labels=["Malignant", "Benign"]).plot(
        ax=a, colorbar=False, cmap="Blues"); a.set_title(n)
plt.tight_layout(); plt.savefig(OUT + "confusion.png", dpi=150); plt.show() # 4c.
ROC
plt.figure(figsize=(6.5, 5)) for n
in names:
    plt.plot(*roc_store[n], label=f"{n} (AUC={res.set_index('Model').loc[n,'ROC-AUC']:.3f})") plt.plot([0,
1], [0, 1], "k--"); plt.xlabel("False Positive Rate"); plt.ylabel("True Positive Rate") plt.title("ROC Curves");
plt.legend(loc="lower right"); plt.grid(True)
plt.tight_layout(); plt.savefig(OUT + "roc.png", dpi=150); plt.show() # 4d.
log loss
plt.figure(figsize=(6.5, 4.5))
plt.bar(names, res["Log Loss"], color="tab:red"); plt.ylabel("Log Loss (lower is better)") plt.title("Log Loss
Comparison"); plt.grid(axis="y")
plt.tight_layout(); plt.savefig(OUT + "logloss.png", dpi=150); plt.show() # 4e.
KNN k vs accuracy
ks = range(1, 26); accs = [] for k
in ks:
    p = Pipeline([("sc", StandardScaler()), ("clf", KNeighborsClassifier(k))])
    accs.append(cross_val_score(p, X_train, y_train, cv=cv).mean())
plt.figure(figsize=(6.5, 4.5)); plt.plot(ks, accs, marker="o")
plt.xlabel("k"); plt.ylabel("5-fold CV Accuracy"); plt.title("KNN: Effect of k"); plt.grid(True)
plt.tight_layout(); plt.savefig(OUT + "knn_k.png", dpi=150); plt.show()
# 4f. Decision tree depth: train vs CV
depths = range(1, 11); tr, te = [], [] for d
in depths:
    p = DecisionTreeClassifier(max_depth=d, random_state=RS).fit(X_train, y_train)
```

```
    tr.append(p.score(X_train, y_train)) te.append(cross_val_score(DecisionTreeClassifier(max_depth=d,
    random_state=RS), X_train,
y_train, cv=cv).mean())
plt.figure(figsize=(6.5, 4.5)); plt.plot(depths, tr, marker="o", label="Train") plt.plot(depths, te,
marker="s", label="CV"); plt.xlabel("max_depth"); plt.ylabel("Accuracy") plt.title("Decision Tree:
Depth vs Accuracy"); plt.legend(); plt.grid(True) plt.tight_layout(); plt.savefig(OUT +
"dt_depth.png", dpi=150); plt.show()

# 5. SAVE
out = {"results": results, "best_params": best_params, "cm": {k:
        v.tolist() for k, v in cm_store.items()},
        "knn_cv": dict(zip(ks, accs)), "dt_train": tr, "dt_cv": te, "n_train":
        len(X_train), "n_test": len(X_test)}
json.dump(out, open(OUT + "results.json", "w"), indent=1, default=float)
```

16. Student Declaration

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	PRN	Signature
Atharv Jeevan Fatale	202401070076	Atharvfatale